

PICCP Development Roadmap

Version: 1.0

Last Updated: December 2025

Estimated Timeline to v1.0: 12-15 months

Overview

This roadmap outlines a realistic development timeline for PICCP, accounting for the complexity of cryptographic systems, security hardening, and the need for external review. The phases are designed to be sequential with some overlap, and each includes explicit security and testing milestones.

Phase 0: Specification Finalization

Duration: 3-4 weeks

Team: 1-2 people

Objectives

- Freeze core cryptographic protocol specification
- Define wire formats and message structures
- Specify relay API contracts
- Document test vectors for all cryptographic operations

Deliverables

- ☐ Complete protocol specification (PICCP v1.0 spec)
- ☐ Wire format documentation with test vectors
- ☐ Relay API specification (OpenAPI/Swagger)
- ☐ Threat model refinement
- ☐ Security requirements document

Dependencies

- Community feedback on whitepaper
- Review by at least 2 external cryptographers (informal)

Success Criteria

- No major protocol changes anticipated
- Test vectors verified by independent implementation
- Consensus on core design decisions

Risks

- Scope creep (new feature requests during review)
- Fundamental design flaws discovered late

Contingency: Budget 1 additional week for addressing critical feedback

Phase 1: Cryptographic Core Implementation

Duration: 2.5-3 months

Team: 1-2 people with cryptography experience

Objectives

- Implement ML-KEM and ML-DSA integration
- Build key derivation and management layer
- Implement Double Ratchet state machine
- Develop identity rotation mechanism with continuity proofs
- Create comprehensive test suite

Milestones

Milestone 1.1: Primitive Integration (3 weeks)

- ☐ ML-KEM-768 wrapper with safe API
- ☐ ML-DSA-65 wrapper with safe API
- ☐ BLAKE3 KDF implementation
- ☐ AES-256-GCM wrapper
- ☐ Test vectors passing for all primitives
- ☐ Constant-time operations verification

Milestone 1.2: Key Exchange (3 weeks)

- ☐ Prekey bundle generation and serialization
- ☐ X3DH-style initialization protocol
- ☐ Session establishment logic

- ☐ Initial key derivation
- ☐ Test cases: successful exchange, missing prekeys, replays

Milestone 1.3: Double Ratchet (4 weeks)

- ☐ Symmetric chain key ratchet
- ☐ Root key ratchet with ML-KEM
- ☐ Out-of-order message handling
- ☐ Skipped message key storage (bounded)
- ☐ State serialization and recovery
- ☐ Extensive state machine testing (fuzzing)

Milestone 1.4: Identity Rotation (2 weeks)

- ☐ Rotation proof generation
- ☐ Proof verification
- ☐ Rotation counter management
- ☐ Integration with session state

Milestone 1.5: Security Hardening (2-3 weeks)

- ☐ Memory zeroization for sensitive data
- ☐ Side-channel mitigation (timing attacks)
- ☐ Secure random number generation verification
- ☐ Key storage encryption at rest
- ☐ Comprehensive fuzzing (at least 100M iterations)
- ☐ Property-based testing for state machine
- ☐ Code review (internal + 1 external reviewer)

Deliverables

- ☐ `piccp-crypto` library with stable API
- ☐ Test suite with >95% code coverage
- ☐ Fuzzing infrastructure
- ☐ API documentation
- ☐ Security audit report (self + external)

Dependencies

- Specification freeze (Phase 0)
- ML-KEM/ML-DSA reference implementations (available)

Success Criteria

- All test vectors passing
- Fuzzing reveals no crashes or hangs
- External review finds no critical issues
- Performance benchmarks within acceptable range (<5ms per message)

Risks

- **HIGH:** Subtle bugs in ratchet state machine
- **MEDIUM:** Performance issues with PQ primitives
- **MEDIUM:** Memory safety issues

Contingencies:

- Allocate 2 additional weeks for unexpected complexity

- Budget for external cryptographic review (\$2-5K)
-

Phase 2: Relay Implementation

Duration: 6-8 weeks

Team: 1-2 people

Objectives

- Build zero-trust relay server
- Implement capability-based routing
- Deploy epoch bucketing and message storage
- Create relay API endpoints
- Develop admin tools and monitoring

Milestones

Milestone 2.1: Core Relay (3 weeks)

- ☐ Capability-based mailbox creation
- ☐ Message ingestion endpoint
- ☐ Epoch bucketing logic
- ☐ Storage backend (initial: filesystem, later: PostgreSQL)
- ☐ Basic rate limiting

Milestone 2.2: Message Retrieval (2 weeks)

- ☐ Epoch listing endpoint

- ☐ Epoch retrieval with capability auth
- ☐ Padding to fixed message sizes
- ☐ Efficient batch operations

Milestone 2.3: Operational Concerns (2 weeks)

- ☐ Message expiry and cleanup
- ☐ Storage quota management
- ☐ Logging (minimal, privacy-preserving)
- ☐ Monitoring and metrics (aggregate only)
- ☐ Health check endpoints

Milestone 2.4: Hardening (1-2 weeks)

- ☐ DoS protection (rate limiting, size limits)
- ☐ Input validation and sanitization
- ☐ TLS configuration
- ☐ Security headers
- ☐ Load testing (10K+ messages/hour)
- ☐ Failure mode testing

Deliverables

- ☐ `piccp-relay` server application
- ☐ Deployment documentation
- ☐ Admin CLI tools
- ☐ Monitoring dashboards
- ☐ Load testing report

Dependencies

- Finalized relay API spec (Phase 0)
- Cryptographic library for capability generation (Phase 1)

Success Criteria

- Handles 10,000 messages/hour on single instance
- Sub-100ms p99 latency for message submission
- Passes security scanner (e.g., OWASP ZAP)
- Zero metadata leakage confirmed by review

Risks

- **MEDIUM:** Storage scaling issues
- **MEDIUM:** DoS vulnerabilities
- **LOW:** Epoch synchronization bugs

Contingency: Add 1 week for performance optimization

Phase 3: Client Implementation

Duration: 8-10 weeks

Team: 2-3 people

Objectives

- Build CLI client for testing and power users
- Implement relay communication layer
- Create contact management system
- Develop local message storage
- Build basic GUI (optional, stretch goal)

Milestones

Milestone 3.1: CLI Foundations (3 weeks)

- ☐ Identity creation and management
- ☐ Prekey bundle generation and upload
- ☐ Mailbox creation and management
- ☐ Basic send/receive loop
- ☐ CLI interface with clear commands

Milestone 3.2: Contact Management (2 weeks)

- ☐ Contact database (identity keys, mailbox IDs)
- ☐ Key verification workflows
- ☐ Identity rotation UI
- ☐ Trust-on-first-use (TOFU) implementation

Milestone 3.3: Message Handling (2 weeks)

- ☐ Epoch fetching and decryption
- ☐ Message ordering and threading

- ☐ Duplicate detection
- ☐ Local message storage (encrypted)
- ☐ Search and history

Milestone 3.4: User Experience (2 weeks)

- ☐ Notifications for new messages
- ☐ Background epoch polling
- ☐ Error handling and user feedback
- ☐ Configuration management
- ☐ Help documentation

Milestone 3.5: Optional GUI (3 weeks, stretch)

- ☐ Basic Electron or Tauri app
- ☐ Conversation view
- ☐ Contact list
- ☐ Settings panel

Deliverables

- ☐ `piccp-cli` command-line client
- ☐ User documentation and tutorials
- ☐ (Optional) `piccp-gui` desktop application
- ☐ Client library for third-party integrations

Dependencies

- Crypto library (Phase 1)
- Relay deployed and accessible (Phase 2)

Success Criteria

- Can send/receive messages reliably
- Key verification workflow is clear
- Local storage encrypted and safe
- User documentation complete

Risks

- **MEDIUM:** UX complexity confuses users
- **MEDIUM:** Background polling battery impact
- **LOW:** Platform-specific issues

Contingency: Skip GUI for v1.0, defer to v1.1

Phase 4: Integration and Testing

Duration: 6-8 weeks

Team: 2-3 people

Objectives

- End-to-end integration testing
- Multi-client interoperability testing
- Security testing and penetration testing
- Performance benchmarking

- Documentation completion

Milestones

Milestone 4.1: Integration Testing (3 weeks)

- ☐ Multi-client message exchange (Alice ↔ Bob scenarios)
- ☐ Identity rotation with continuity proofs
- ☐ Out-of-order message handling
- ☐ Network failure scenarios
- ☐ Relay failure and recovery
- ☐ Message expiry testing

Milestone 4.2: Security Testing (2 weeks)

- ☐ Penetration testing (relay and client)
- ☐ Replay attack testing
- ☐ Man-in-the-middle testing
- ☐ Side-channel analysis (timing, cache)
- ☐ Dependency security audit
- ☐ Supply chain verification

Milestone 4.3: Performance Testing (1 week)

- ☐ End-to-end latency measurement
- ☐ Bandwidth usage analysis
- ☐ Client CPU and memory profiling
- ☐ Relay scaling tests
- ☐ Battery usage on mobile (if applicable)

Milestone 4.4: Documentation (2 weeks)

- ☐ User guide (installation, setup, usage)
- ☐ Developer documentation (API, architecture)
- ☐ Deployment guide (relay setup, hardening)
- ☐ Security best practices
- ☐ FAQ and troubleshooting

Deliverables

- ☐ Integration test suite
- ☐ Security audit report (internal + external review)
- ☐ Performance benchmark report
- ☐ Complete documentation set
- ☐ Known issues and limitations document

Dependencies

- All previous phases complete

Success Criteria

- All integration tests passing
- No critical security issues
- Performance within acceptable bounds
- Documentation reviewed and approved

Risks

- **HIGH:** Critical bugs discovered late
- **MEDIUM:** Performance below requirements

- **MEDIUM:** Security vulnerabilities found

Contingency: Budget 2-4 additional weeks for critical fixes

Phase 5: External Security Review

Duration: 4-6 weeks (mostly waiting)

Budget: \$10-25K

Objectives

- Engage professional security auditors
- Address findings from external review
- Obtain independent validation of security claims

Activities

- ☐ Select reputable security firm (e.g., NCC Group, Trail of Bits, Cure53)
- ☐ Provide codebase and documentation
- ☐ Respond to auditor questions
- ☐ Fix identified vulnerabilities
- ☐ Re-review critical fixes
- ☐ Publish audit report (redacted if necessary)

Deliverables

- ☐ Professional security audit report
- ☐ Fixes for all critical/high findings
- ☐ Public summary of audit (for transparency)

Success Criteria

- No critical or high-severity findings remaining
- Auditors validate core security claims
- Audit report publishable

Risks

- **HIGH:** Critical flaws requiring protocol changes
- **MEDIUM:** Budget constraints limit audit scope

Contingency: If critical flaws found, budget 4-8 additional weeks for fixes and re-audit

Phase 6: Private Beta

Duration: 6-8 weeks

Team: 2-3 people + community testers

Objectives

- Deploy to limited set of users (50-200)
- Gather real-world usage feedback
- Monitor for bugs and edge cases
- Iterate on UX based on feedback

Activities

- ☐ Invite technically savvy early adopters
- ☐ Set up support channels (forum, chat)
- ☐ Monitor relay metrics and logs
- ☐ Collect bug reports and feature requests
- ☐ Release bi-weekly updates
- ☐ Conduct user interviews

Deliverables

- ☐ Bug fixes for issues found in beta
- ☐ UX improvements based on feedback
- ☐ Stability and reliability metrics
- ☐ Beta testing report

Success Criteria

- <1% message delivery failure rate
- No critical bugs in 4 consecutive weeks
- Positive user feedback on core functionality
- At least 500 messages exchanged successfully

Risks

- **MEDIUM:** Major bugs discovered in production
- **LOW:** Low user adoption in beta

Contingency: Extend beta by 2-4 weeks if stability issues persist

Phase 7: Public v1.0 Launch

Duration: 2-3 weeks

Team: Full team + marketing/community

Objectives

- Public release of PICCP v1.0
- Announce to wider audience
- Provide clear onboarding and documentation
- Establish community governance

Activities

- ☐ Final release candidate testing
- ☐ Launch announcement (blog post, social media)
- ☐ Submit to Hacker News, Reddit, crypto forums
- ☐ Host AMA or launch webinar
- ☐ Monitor launch metrics
- ☐ Respond to community questions

Deliverables

- ☐ PICCP v1.0 release (clients + relay)
- ☐ Launch blog post
- ☐ Press kit and media outreach
- ☐ Community forum/Discord setup
- ☐ Post-launch retrospective

Success Criteria

- Stable release with no show-stopper bugs
- Positive reception in crypto/privacy communities
- Clear roadmap for v1.1 and beyond

Timeline Summary

Phase	Duration	Start	End
0: Specification	3-4 weeks	Week 0	Week 4
1: Crypto Core	2.5-3 months	Week 4	Week 16
2: Relay	6-8 weeks	Week 16	Week 24
3: Client	8-10 weeks	Week 20	Week 30
4: Integration Testing	6-8 weeks	Week 30	Week 38
5: External Review	4-6 weeks	Week 38	Week 44
6: Private Beta	6-8 weeks	Week 44	Week 52
7: Public Launch	2-3 weeks	Week 52	Week 55

Total Duration: 12-15 months (55-65 weeks)

Note: Some phases overlap (e.g., Client and Relay development)

Resource Requirements

Team Composition

- **Lead Developer/Cryptographer:** 1 FTE for entire project
- **Backend Developer** (relay): 0.5 FTE for 3 months
- **Frontend/Client Developer:** 0.5-1 FTE for 3 months
- **Security Reviewer/Tester:** 0.25 FTE ongoing
- **Technical Writer:** 0.25 FTE for 2 months

Total: ~1.5-2 FTE-years

Budget (Rough Estimates)

- **Personnel:** \$150-250K (depends on location, experience)
- **Security Audit:** \$10-25K
- **Infrastructure:** \$2-5K (relay hosting, CI/CD)
- **Miscellaneous:** \$3-5K (domains, tools, etc.)

Total: \$165-285K

Success Metrics

Technical Metrics (v1.0)

- ☐ 100% of core test vectors passing
- ☐ >95% code coverage for crypto core
- ☐ Zero critical security vulnerabilities
- ☐ <5ms median message encryption/decryption
- ☐ <100ms p99 relay latency
- ☐ Successfully exchange 10,000+ messages in beta

Adoption Metrics (6 months post-launch)

- ☐ 500+ active users
- ☐ 5+ independent relay deployments
- ☐ 2+ third-party client implementations
- ☐ Featured in at least 3 security/privacy publications

Community Metrics

- ☐ Active developer community (10+ contributors)
- ☐ Regular releases (monthly patches, quarterly features)
- ☐ Clear governance model
- ☐ Responsive to security reports (<24hr acknowledgment)

Post-v1.0 Roadmap

v1.1 (3-4 months post-launch)

- Mobile clients (iOS, Android)

- Improved UX and onboarding
- Multi-device support
- Contact discovery (privacy-preserving)

v1.2 (6-8 months post-launch)

- Proxy routing for enhanced metadata protection
- Relay federation (experimental)
- File attachments
- Message reactions and threading

v2.0 (12-18 months post-launch)

- PIR-based message retrieval
- Small group messaging (2-50 members)
- Decentralized relay network
- Formal security proofs published

v3.0 (18-24 months post-launch)

- Mixnet integration
 - Cover traffic strategies
 - Advanced traffic analysis resistance
 - IETF standardization submission
-

Risk Management

Critical Risks

Risk: Fundamental cryptographic flaw discovered

Probability: Low

Impact: Catastrophic

Mitigation: External review, conservative design, multiple reviews

Response: Immediate disclosure, protocol revision, coordinated upgrade

Risk: Post-quantum primitives broken/weakened

Probability: Very Low

Impact: High

Mitigation: Use NIST-standardized algorithms, monitor research

Response: Rapid migration to alternative primitives

Risk: Team capacity insufficient

Probability: Medium

Impact: High (delays)

Mitigation: Realistic timeline, buffer time, clear milestones

Response: Reduce scope, extend timeline, seek additional help

Risk: Low user adoption

Probability: Medium

Impact: Medium

Mitigation: Focus on usability, clear value proposition, community building

Response: Iterate on UX, target specific use cases, partnerships

High Risks

Risk: Security vulnerabilities in implementation

Probability: Medium

Impact: High

Mitigation: Testing, fuzzing, external audit, bug bounty

Response: Rapid patching, coordinated disclosure, transparency

Risk: Relay scaling issues

Probability: Medium

Impact: Medium

Mitigation: Load testing, efficient design, horizontal scaling

Response: Performance optimization, infrastructure upgrades

Risk: Regulatory challenges (encryption backdoors, etc.)

Probability: Low-Medium

Impact: High

Mitigation: Legal review, jurisdiction selection

Response: Community coordination, advocacy, compliance where possible

Governance and Decision Making

Pre-v1.0

- Lead developer has final say on technical decisions
- Major decisions discussed with advisors/community
- Security issues have override authority

Post-v1.0

- Establish steering committee (3-5 members)
- RFC process for protocol changes
- Security committee for vulnerability handling

- Community voting on major features
-

Contact and Updates

- **Project Lead:** Luiz Widmer
 - **Repository:** github.com/lwidmer/piccp (forthcoming)
 - **Discussion Forum:** TBD
 - **Security Contact:** security@piccp.org (forthcoming)
 - **Monthly Progress Updates:** Blog + mailing list
-

Last Updated: December 17, 2025

Next Review: End of Phase 0 (January 2026)